RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Name:Palachuru Nandu Priya

Roll.no:241801195

Ex1: Data Preprocessing and Cleaning

Problem Statement:

A telecommunications company is experiencing customer churn (customers leaving the service). Before building a machine learning model to predict churn, the data science team must clean and preprocess the customer data to ensure high-quality input for analysis.

You are a Data Scientist working for the company. Your task is to preprocess the customer dataset by handling missing values, removing inconsistencies, transforming categorical features, scaling numerical attributes, and visualizing important relationships.

Learning Objective:

The objective of this experiment is to enable students to understand and perform the essential steps involved in data preprocessing. This experiment will provide a strong foundation for preparing high-quality data for subsequent machine learning and predictive analytics tasks.

Tasks to Perform:

1. Import the required Python libraries.
2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.
3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records
4. Clean the data
    a. Remove duplicate customer records.
    b. Replace missing values
        i. Numerical columns → Median
        ii. Categorical columns → Mode
    c. Verify that no missing values remain.
5. Encode categorical columns: Gender, Partner, Dependents, Contract, PaymentMethod, InternetService, Churn. Use appropriate encoding techniques. 6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.

7. Plot Distribution of MonthlyCharges, Distribution of Tenure.
8. Create a Count Plot showing Contract vs Churn.
9. Draw a Box Plot of MonthlyCharges grouped by Churn.
10. Display the Correlation Heatmap of all numerical features.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
df = pd.read_csv("Telco-Customer-Churn.csv")

print("Dataset Imported Successfully")

print("\nFirst 10 Records")
print(df.head(10))

print("\nLast 10 Records")
print(df.tail(10))

print("\nShape of Dataset")
print(df.shape)

print("\nColumn Names")
print(df.columns)
```

```
Dataset Imported Successfully

First 10 Records
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes
5  9305-CDSKC  Female              0      No         No       8          Yes
6  1452-KIOVK    Male              0      No        Yes      22          Yes
7  6713-OKOMC  Female              0      No         No      10           No
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
```

```
print("\nDataset Information")
print(df.info())

print("\nSummary Statistics")
print(df.describe())

print("\nMissing Values")
print(df.isnull().sum())

print("\nDuplicate Records")
print(df.duplicated().sum())
```

```
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

Summary Statistics
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000

Missing Values
customerID          0
gender              0
SeniorCitizen       0
Partner             0
```

```python
#Palachuru Nandu Priya [241801195]
#step 4a
print("Duplicate Records Before:", df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("Duplicate Records After:", df.duplicated().sum())
```

```
Duplicate Records Before: 0
Duplicate Records After: 0
```

```python
numerical_columns = df.select_dtypes(include=["int64","float64"]).columns

for col in numerical_columns:
    df[col].fillna(df[col].median(), inplace=True)

    #step 4b
    categorical_columns = df.select_dtypes(include=["object"]).columns

for col in categorical_columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
/tmp/ipykernel_1735/3491636176.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1735/3491636176.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```python
#Palachuru Nandu Priya [241801195]

print("Missing Values After Cleaning")
print(df.isnull().sum())
```

```
Missing Values After Cleaning
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```python
#Palachuru Nandu Priya [241801195]

le = LabelEncoder()

columns = ["gender","Partner","Dependents","Contract",
           "PaymentMethod","InternetService","Churn"]

for col in columns:
    df[col] = le.fit_transform(df[col])

print("Encoding Completed")
print(df[columns].head())
```

```
Encoding Completed
   gender  Partner  Dependents  Contract  PaymentMethod  InternetService  \
0       0        1           0         0              2                0
1       1        0           0         1              3                0
2       1        0           0         0              3                0
3       1        0           0         1              0                0
4       0        0           0         0              2                1

   Churn
0      0
1      0
2      1
3      0
4      1
```

```python
scaler = StandardScaler()

cols = ["tenure","MonthlyCharges","TotalCharges"]

# Convert 'TotalCharges' to numeric, coercing errors to NaN
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
# Fill any NaN values that resulted from coercion with the median
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

df[cols] = scaler.fit_transform(df[cols])

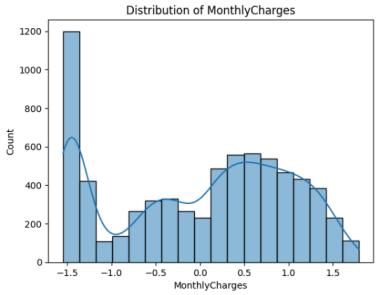
print(df[cols].head())
```

```
      tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
/tmp/ipykernel_1735/1334993591.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

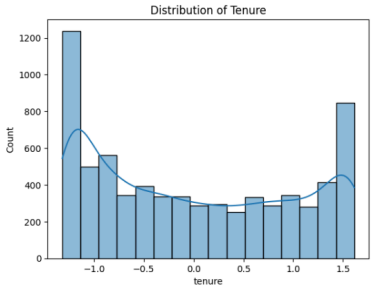
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
```

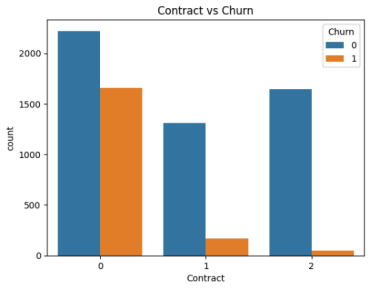
```
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of MonthlyCharges")
plt.show()
```




```
sns.histplot(df["tenure"], kde=True)
plt.title("Distribution of Tenure")
plt.show()
```



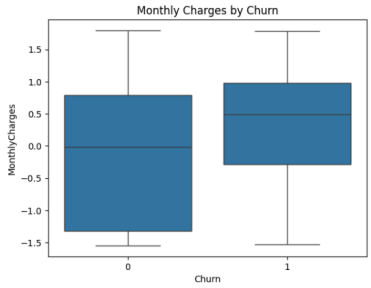

```
sns.countplot(x="Contract", hue="Churn", data=df)
plt.title("Contract vs Churn")
plt.show()
```



```
sns.boxplot(x="Churn", y="MonthlyCharges", data=df)

plt.title("Monthly Charges by Churn")

plt.show()
```

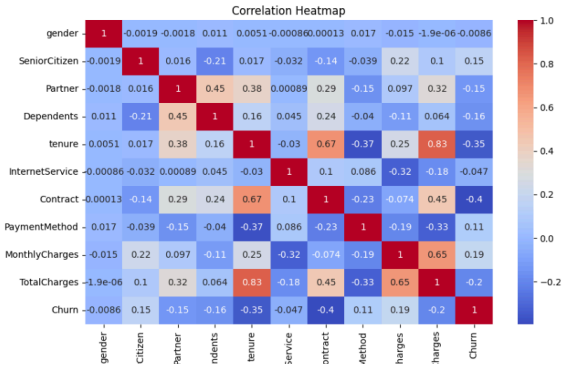



```
plt.figure(figsize=(10,6))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm")

plt.title("Correlation Heatmap")

plt.show()
```



Ex2: Data Discovery and Preparation

Learning Objective:
The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Question:
A telecommunications company has observed an increase in customer churn, leading to revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset, identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform:

1. Reuse Ex:1 Steps 1- 6, 10.
2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method. 3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms and Boxplots.
4. Detect outliers in numerical features using the Interquartile Range (IQR) method. 5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study the relationship between: Contract Type and Churn, Internet Service and Churn, Monthly Charges and Churn, Tenure and Churn.
6. Do the below analytical questions:
   - Which contract type has the highest customer churn?
   - Which payment method is most frequently used by customers who churn? • Do customers with shorter tenure tend to churn more often?
   - Which customer group has the highest monthly charges?
   - Which Internet Service category experiences the highest churn?
   - Is there any relationship between monthly charges and churn?
   - Which features appear to be the strongest indicators of customer churn? 7.

Split the cleaned dataset into Training (80%) and Testing (20%) datasets. 8. Save the final processed dataset as a new CSV file named CustomerChurn_Preprocessed.csv.

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 1

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 2

df = pd.read_csv(" Customer_Churn.csv")

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 3

print(df.shape)

df.info()

df.describe()
```

```
(891, 12)
```

```
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 4

imputer = SimpleImputer(strategy='mean')

df['Age'] = imputer.fit_transform(df[['Age']])
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 5
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 5

df['Cabin'] = df['Cabin'].fillna("Unknown")
```

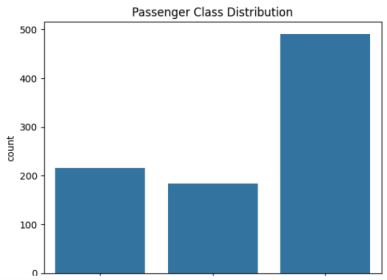
```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 6

mode_value = df['Embarked'].mode()[0]

df['Embarked'] = df['Embarked'].fillna(mode_value)
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 7

sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Distribution")
plt.show()
```




```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 8

female_survived = df[(df['Sex']=="female") & (df['Survived']==1)]

print(female_survived['Name'])
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
2                  Heikkinen, Miss. Laina
3            Futrelle, Mrs. Jacques Heath (Lily May Peel)
8      Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                    Nasser, Mrs. Nicholas (Adele Achem)
                             ...
874               Abelson, Mrs. Samuel (Hannah Wizosky)
875                    Najib, Miss. Adele Kiamie "Jane"
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880          Shelley, Mrs. William (Imanita Parrish Hall)
887                      Graham, Miss. Margaret Edith
Name: Name, Length: 233, dtype: object
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 9

result = df[(df['Pclass']==3) & (df['Age']<18)]

print(result)
```

```
     PassengerId  Survived  Pclass                                          Name  \
7              8         0       3                Palsson, Master. Gosta Leonard
10            11         1       3               Sandstrom, Miss. Marguerite Rut
14            15         0       3          Vestrom, Miss. Hulda Amanda Adolfina
16            17         0       3                         Rice, Master. Eugene
22            23         1       3                    McGowan, Miss. Anna "Annie"
..           ...       ...     ...                                           ...
844          845         0       3                          Culumovic, Mr. Jeso
850          851         0       3  Andersson, Master. Sigvard Harald Elias
852          853         0       3                       Boulos, Miss. Nourelain
869          870         1       3               Johnson, Master. Harold Theodor
875          876         1       3              Najib, Miss. Adele Kiamie "Jane"

        Sex   Age  SibSp  Parch   Ticket     Fare    Cabin Embarked
7      male   2.0      3      1   349909  21.0750  Unknown        S
10   female   4.0      1      1  PP 9549  16.7000       G6        S
14   female  14.0      0      0   350406   7.8542  Unknown        S
16     male   2.0      4      1   382652  29.1250  Unknown        Q
22   female  15.0      0      0   330923   8.0292  Unknown        Q
..      ...   ...    ...    ...      ...      ...      ...      ...
844    male  17.0      0      0   315090   8.6625  Unknown        S
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 10

result = df[(df['Pclass']==1) & (df['Age']>40)]

print(result)
```

```
     PassengerId  Survived  Pclass  \
6              7         0       1
11            12         1       1
35            36         0       1
52            53         1       1
54            55         0       1
..           ...       ...     ...
856          857         1       1
857          858         1       1
862          863         1       1
871          872         1       1
879          880         1       1

                                                  Name     Sex   Age  SibSp  \
6                              McCarthy, Mr. Timothy J    male  54.0      0
11                            Bonnell, Miss. Elizabeth  female  58.0      0
35                         Holverson, Mr. Alexander Oskar    male  42.0      1
52               Harper, Mrs. Henry Sleeper (Myna Haxtun)  female  49.0      1
54                         Ostby, Mr. Engelhart Cornelius    male  65.0      0
..                                                    ...     ...   ...    ...
856           Wick, Mrs. George Dennick (Mary Hitchcock)  female  45.0      1
857                               Daly, Mr. Peter Denis    male  51.0      0
862    Swift, Mrs. Frederick Joel (Margaret Welles Ba...  female  48.0      0
871     Beckwith, Mrs. Richard Leonard (Sallie Monypeny)  female  47.0      1
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)  female  56.0      0

     Parch    Ticket      Fare    Cabin Embarked
6        0     17463   51.8625      E46        S
11       0    113783   26.5500     C103        S
35       0    113789   52.0000  Unknown        S
52       0  PC 17572   76.7292      D33        C
54       1    113509   61.9792      B30        C
..     ...       ...       ...      ...      ...
856      1     36928  164.8667  Unknown        S
857      0    113055   26.5500      E17        S
862      0     17466   25.9292      D17        S
871      1     11751   52.5542      D35        S
879      1     11767   83.1583      C50        C

[76 rows x 12 columns]
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 11
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 11

result = df[(df['Pclass']==1) &
            (df['Age']>40) &
            (df['Survived']==1)]

print(result)
```

```
                                                  Name     Sex   Age  SibSp  \
621                          Kimball, Mr. Edwin Nelson Jr    male  42.0      1
630               Barkworth, Mr. Algernon Henry Wilson    male  80.0      0
645                           Harper, Mr. Henry Sleeper    male  48.0      1
647                  Simonius-Blumer, Col. Oberst Alfons    male  56.0      0
660                       Frauenthal, Dr. Henry William    male  50.0      2
707                    Calderhead, Mr. Edward Pennington    male  42.0      0
712                              Taylor, Mr. Elmer Zebley    male  48.0      1
765                  Hogeboom, Mrs. John C (Anna Andrews)  female  51.0      1
779    Robert, Mrs. Edward Scott (Elisabeth Walton Mc...  female  43.0      0
796                          Leader, Dr. Alice (Farnham)  female  49.0      0
820    Hays, Mrs. Charles Melville (Clara Jennings Gr...  female  52.0      1
829           Stone, Mrs. George Nelson (Martha Evelyn)  female  62.0      0
856           Wick, Mrs. George Dennick (Mary Hitchcock)  female  45.0      1
857                               Daly, Mr. Peter Denis    male  51.0      0
862    Swift, Mrs. Frederick Joel (Margaret Welles Ba...  female  48.0      0
871     Beckwith, Mrs. Richard Leonard (Sallie Monypeny)  female  47.0      1
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)  female  56.0      0

     Parch    Ticket      Fare    Cabin Embarked
11       0    113783   26.5500     C103        S
52       0  PC 17572   76.7292      D33        C
187      0    111428   26.5500  Unknown        S
194      0  PC 17610   27.7208       B4        C
195      0  PC 17569  146.5208      B80        C
268      1  PC 17582  153.4625     C125        S
275      0     13502   77.9583       D7        S
299      1  PC 17558  247.5208  B58 B60        C
337      0     16966  134.5000      E40        C
366      0    110813   75.2500      D37        C
380      0  PC 17757  227.5250  Unknown        C
449      0    113786   30.5000     C104        S
453      0     17453   89.1042      C92        C
460      0     19952   26.5500      E12        S
496      0     36947   78.2667      D20        C
513      0  PC 17603   59.4000  Unknown        C
523      1    111361   57.9792      B18        C
556      0     11755   39.6000      A16        C
571      0     11769   51.4792     C101        S
587      1     13567   79.2000      B41        C
591      0     36947   78.2667      D20        C
599      0  PC 17485   56.9292      A20        C
621      0     11753   52.5542      D19        S
630      0     27042   30.0000      A23        S
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 12

result = df[(df['Sex']=="male") &
            (df['Fare']>100)]

print(result[['Name','Fare']])
```

```
                                          Name      Fare
27             Fortune, Mr. Charles Alexander  263.0000
118                 Baxter, Mr. Quigg Edmond  247.5208
305            Allison, Master. Hudson Trevor  151.5500
332                 Graham, Mr. George Edward  153.4625
373                     Ringhini, Mr. Sante  135.6333
377                  Widener, Mr. Harry Elkins  211.5000
390                Carter, Mr. William Ernest  120.0000
438                         Fortune, Mr. Mark  263.0000
505  Penasco y Castellana, Mr. Victor de Satode  108.9000
527                       Farthing, Mr. John  221.7792
544                  Douglas, Mr. Walter Donald  106.4250
550                Thayer, Mr. John Borland Jr  110.8833
557                       Robbins, Mr. Victor  227.5250
659                  Newell, Mr. Arthur Webster  113.2750
660               Frauenthal, Dr. Henry William  133.6500
679          Cardeza, Mr. Thomas Drake Martinez  512.3292
698                   Thayer, Mr. John Borland  110.8833
737                     Lesurer, Mr. Gustave J  512.3292
802         Carter, Master. William Thornton II  120.0000
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 13

result = df[(df['Embarked']=="C") &
            (df['Pclass']==2)]

print(result)
```

```
     PassengerId  Survived  Pclass  \
9             10         1       2
43            44         1       2
122          123         0       2
135          136         0       2
181          182         0       2
292          293         0       2
308          309         0       2
361          362         0       2
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 14

result = df[df['SibSp']>2]

print(result)
```

```
     PassengerId  Survived  Pclass  \
7              8         0       3
16            17         0       3
24            25         0       3
27            28         0       1
50            51         0       3
59            60         0       3
63            64         0       3
68            69         1       3
71            72         0       3
85            86         1       3
88            89         1       1
119          120         0       3
159          160         0       3
164          165         0       3
171          172         0       3
176          177         0       3
180          181         0       3
182          183         0       3
201          202         0       3
229          230         0       3
233          234         1       3
261          262         1       3
266          267         0       3
278          279         0       3
324          325         0       3
341          342         1       1
374          375         0       3
386          387         0       3
409          410         0       3
480          481         0       3
485          486         0       3
541          542         0       3
542          543         0       3
634          635         0       3
642          643         0       3
683          684         0       3
686          687         0       3
726          727         1       2
787          788         0       3
792          793         0       3
813          814         0       3
819          820         0       3
824          825         0       3
```

```
#Palachuru Nandu Priya [241801195]
#EX-2
#step 15

result = df[(df['Survived']==0) &
            (df['SibSp']==0) &
            (df['Parch']==0)]

print(result)
```

```
     PassengerId  Survived  Pclass                                Name  \
4              5         0       3             Allen, Mr. William Henry
5              6         0       3                     Moran, Mr. James
6              7         0       1              McCarthy, Mr. Timothy J
12            13         0       3       Saundercock, Mr. William Henry
14            15         0       3  Vestrom, Miss. Hulda Amanda Adolfina
..           ...       ...     ...                                  ...
882          883         0       3          Dahlberg, Miss. Gerda Ulrika
883          884         0       2         Banfield, Mr. Frederick James
884          885         0       3                Sutehall, Mr. Henry Jr
886          887         0       2                  Montvila, Rev. Juozas
890          891         0       3                   Dooley, Mr. Patrick

        Sex        Age  SibSp  Parch            Ticket     Fare    Cabin  \
4      male  35.000000      0      0            373450   8.0500  Unknown
5      male  29.699118      0      0            330877   8.4583  Unknown
6      male  54.000000      0      0             17463  51.8625      E46
12     male  20.000000      0      0         A/5. 2151   8.0500  Unknown
14   female  14.000000      0      0            350406   7.8542  Unknown
..      ...        ...    ...    ...               ...      ...      ...
882  female  22.000000      0      0              7552  10.5167  Unknown
883    male  28.000000      0      0  C.A./SOTON 34068  10.5000  Unknown
884    male  25.000000      0      0   SOTON/OQ 392076   7.0500  Unknown
886    male  27.000000      0      0            211536  13.0000  Unknown
890    male  32.000000      0      0            370376   7.7500  Unknown

    Embarked
4          S
5          Q
6          S
12         S
14         S
..       ...
882        S
883        S
884        S
886        S
890        Q

[374 rows x 12 columns]
```

```python
#Palachuru Nandu Priya [241801195]
#EX-2
#step 16

result = df[df['Survived']==1]

result = result.sort_values(by='Age', ascending=False)

print(result.head(5))
```

```
     PassengerId  Survived  Pclass                                     Name  \
630          631         1       1       Barkworth, Mr. Algernon Henry Wilson
275          276         1       1          Andrews, Miss. Kornelia Theodosia
483          484         1       3                     Turkula, Mrs. (Hedwig)
570          571         1       2                            Harris, Mr. George
829          830         1       1  Stone, Mrs. George Nelson (Martha Evelyn)

        Sex   Age  SibSp  Parch       Ticket     Fare    Cabin Embarked
630    male  80.0      0      0        27042  30.0000      A23        S
275  female  63.0      1      0        13502  77.9583       D7        S
483  female  63.0      0      0         4134   9.5875  Unknown        S
570    male  62.0      0      0  S.W./PP 752  10.5000  Unknown        S
829  female  62.0      0      0       113572  80.0000      B28        S
```

```python
#Palachuru Nandu Priya [241801195]
#EX-2
#step 17

result = df[df['Fare']==0]

print(result[['Name','Fare']])
```

```
                                Name  Fare
179              Leonard, Mr. Lionel   0.0
263           Harrison, Mr. William    0.0
271      Tornquist, Mr. William Henry   0.0
277          Parkes, Mr. Francis "Frank"   0.0
302  Johnson, Mr. William Cahoone Jr   0.0
413    Cunningham, Mr. Alfred Fleming   0.0
466              Campbell, Mr. William   0.0
481  Frost, Mr. Anthony Wood "Archie"   0.0
597                Johnson, Mr. Alfred   0.0
633     Parr, Mr. William Henry Marsh   0.0
674        Watson, Mr. Ennis Hastings   0.0
732            Knight, Mr. Robert J   0.0
```

```python
#Palachuru Nandu Priya [241801195]
#EX-2
#step 18

X = df.drop('Survived', axis=1)
y = df['Survived']

X = pd.get_dummies(X)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(712, 1731)
(179, 1731)
```

Colab paid products - Cancel contracts here

Ex3a: Model Planning and Building - Classification

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate basic machine learning models using real-world datasets.

Problem Statement:
A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After completing data preprocessing and exploratory analysis, the company now aims to develop predictive models that can identify customers who are likely to churn. Early prediction of customer churn will enable the company to implement targeted retention strategies, improve customer satisfaction, and reduce business losses.

As a Data Scientist, your task is to build and compare multiple classification models to predict whether a customer will churn. You will train different machine learning algorithms, evaluate their performance using appropriate classification metrics, and identify the most effective model for customer churn prediction. Finally, interpret the important factors influencing customer churn and communicate your findings through suitable visualizations and performance comparisons.

Tasks to Perform:

1. Use CustomerChurn_Preprocessed.csv file which you created in Ex2. 2. Split the cleaned dataset into Training (80%) and Testing (20%) datasets. **Model Building**
3. Train a Logistic Regression classifier and predict the customer churn on the test dataset. 4. Train a Decision Tree Classifier and predict the customer churn on the test dataset. 5. Train a Random Forest Classifier and predict the customer churn on the test dataset. **Model Evaluation**
6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score. 7. Compare the performance of all three models using a single bar chart showing the evaluation metrics.
8. Identify the best-performing model based on the evaluation results.

Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[6]
```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)
```

[8]
```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

from google.colab import files

uploaded = files.upload()
df = pd.read_csv('CustomerChurn_Preprocessed.csv')

print("Dataset Shape:", df.shape)
display(df.head())
df.info()
```

```
Choose Files CustomerC...ocessed.csv
CustomerChurn_Preprocessed.csv(text/csv) - 3282220 bytes, last modified: 10/13/2026 - 100% done
Saving CustomerChurn_Preprocessed.csv to CustomerChurn_Preprocessed.csv
Dataset Shape: (64374, 12)
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	1	22	Female	25	14	4	27	Basic	Monthly	598	9	1
1	2	41	Female	28	28	7	13	Standard	Monthly	584	20	0
2	3	47	Male	27	10	2	29	Premium	Annual	757	21	0
3	4	35	Male	9	12	5	17	Premium	Quarterly	232	18	0
4	5	53	Female	58	24	9	2	Standard	Annual	533	18	0

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 12 columns):
```

Variables Terminal

10:24 PM Python 3

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[9]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

df = df.drop(columns=['customerID'], errors='ignore')

if df['Churn'].dtype == 'object':
    df['Churn'] = df['Churn'].astype(str).str.strip().map(
        {'Yes': 1, 'No': 0}
    )

df = df.dropna(subset=['Churn'])

X = pd.get_dummies(df.drop(columns=['Churn']), drop_first=True)
y = df['Churn'].astype(int)

X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())

print("Features:", X.shape)
print("Target Distribution:\n", y.value_counts())
```

```
Features: (64374, 13)
Target Distribution:
 Churn
0    33881
1    30493
Name: count, dtype: int64
```

[10]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (51499, 13)
Testing Data: (12875, 13)
```

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[11]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, random_state=42)
)

lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

print("Logistic Regression")
print(classification_report(y_test, lr_pred, zero_division=0))
```

```
Logistic Regression
              precision    recall  f1-score   support

           0       0.87      0.87      0.87      6776
           1       0.85      0.85      0.85      6099

    accuracy                           0.86     12875
   macro avg       0.86      0.86      0.86     12875
weighted avg       0.86      0.86      0.86     12875
```

[12]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

dt = DecisionTreeClassifier(max_depth=10, random_state=42)

dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

print("Decision Tree")
print(classification_report(y_test, dt_pred, zero_division=0))
```

```
Decision Tree
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      6776
           1       1.00      1.00      1.00      6099

    accuracy                           1.00     12875
   macro avg       1.00      1.00      1.00     12875
weighted avg       1.00      1.00      1.00     12875
```

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[13]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("Random Forest")
print(classification_report(y_test, rf_pred, zero_division=0))
```

```
Random Forest
              precision    recall  f1-score   support

           0       0.99      1.00      1.00      6776
           1       1.00      0.99      1.00      6099

    accuracy                           1.00     12875
   macro avg       1.00      1.00      1.00     12875
weighted avg       1.00      1.00      1.00     12875
```

[14]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

models = {
    'Logistic Regression': lr_pred,
    'Decision Tree': dt_pred,
    'Random Forest': rf_pred
}

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, pred) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, pred, labels=[0, 1])
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=['No Churn', 'Churn'],
        yticklabels=['No Churn', 'Churn'], ax=ax
    )
    ax.set_title(name)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
```

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

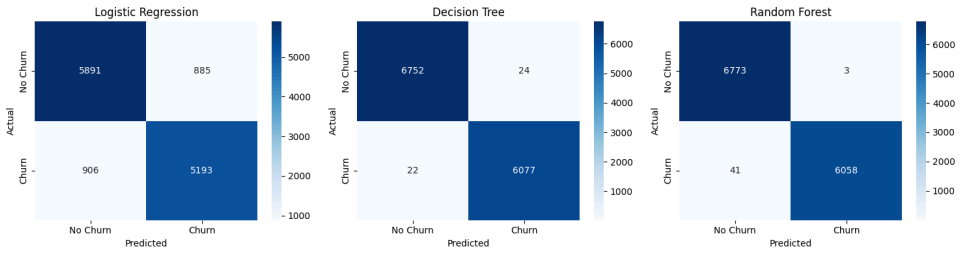
Share

Commands + Code + Text Run all

RAM Disk

[14]

```
plt.tight_layout()
plt.show()
```



pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[15]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

results = []

for name, pred in models.items():
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred, zero_division=0),
        'Recall': recall_score(y_test, pred, zero_division=0),
        'F1-Score': f1_score(y_test, pred, zero_division=0)
    })

results_df = pd.DataFrame(results).set_index('Model')
display(results_df.round(4))

results_df.plot(kind='bar', figsize=(12, 6), ylim=(0, 1))
plt.title('Classification Model Performance Comparison')
plt.ylabel('Score')
plt.xticks(rotation=15)
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

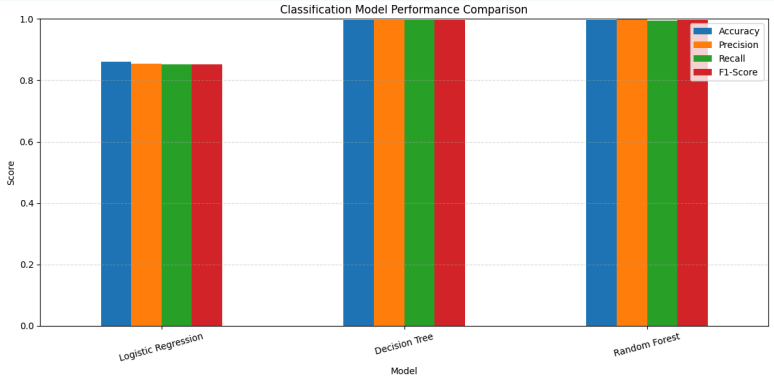
best_model_name = results_df['F1-Score'].idxmax()
print("Best Model:", best_model_name)
```

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8609	0.8544	0.8515	0.8529
Decision Tree	0.9964	0.9961	0.9964	0.9962
Random Forest	0.9966	0.9995	0.9933	0.9964

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all



pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[16]
# Name: Palachuru Nandu Priya
# Roll No: 241801195

best_model = {
    'Logistic Regression': lr,
    'Decision Tree': dt,
    'Random Forest': rf
}[best_model_name]

if best_model_name == 'Logistic Regression':
    importance = np.abs(
        best_model.named_steps['logisticregression'].coef_[0]
    )
else:
    importance = best_model.feature_importances_

feature_importance = pd.Series(
    importance, index=X.columns
).sort_values(ascending=False)

print("Top 10 Important Features:")
print(feature_importance.head(10))

feature_importance.head(10).sort_values().plot(
    kind='barh', figsize=(9, 5)
)
plt.title('Top 10 Features Influencing Customer Churn')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

print("\nBusiness Recommendations:")
print("1. Offer targeted discounts and loyalty benefits.")
print("2. Improve customer support and resolve service complaints.")
print("3. Provide suitable plans to customers at high risk of churn.")
```

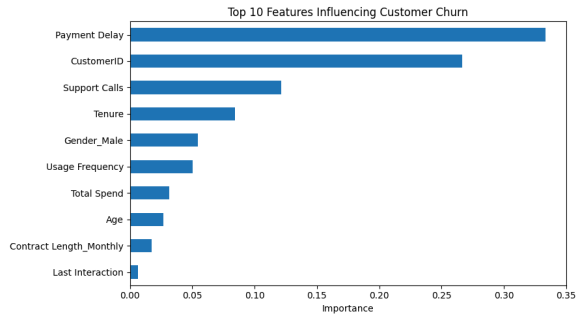
```
Top 10 Important Features:
Payment Delay              0.333511
CustomerID                 0.266802
Support Calls              0.121568
Tenure                     0.084546
Gender_Male                0.054316
Usage Frequency            0.050593
Total Spend                0.031569
Age                        0.026718
Contract Length_Monthly    0.017284
Last Interaction           0.006582
dtype: float64
```

pods-exp-3a.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Top 10 Important Features:
Payment Delay              0.333511
CustomerID                 0.266802
Support Calls              0.121568
Tenure                     0.084546
Gender_Male                0.054316
Usage Frequency            0.050593
Total Spend                0.031569
Age                        0.026718
Contract Length_Monthly    0.017284
Last Interaction           0.006582
dtype: float64
```




```
Business Recommendations:
1. Offer targeted discounts and loyalty benefits.
2. Improve customer support and resolve service complaints.
3. Provide suitable plans to customers at high risk of churn.
```

+ Code + Text

Ex3b: Model Planning and Building - Regression

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate regression models using a real-world dataset.

Problem Statement:
A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels—TV, Radio, and Newspaper—and wants to understand how these advertising expenditures influence product sales. By developing predictive models, the company aims to estimate future sales based on advertising investments and identify the most suitable regression model for accurate sales prediction.

As a Data Scientist, your task is to build and compare multiple regression models using the Advertising dataset. Evaluate the performance of each model using appropriate regression metrics and communicate the findings through visualizations to support business decision-making.

Tasks to Perform:
1. Download the Advertising Dataset from Kaggle and load it into a Pandas DataFrame. 2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics. 3. Select: TV, Radio, Newspaper as Input Features (X), Sales as Target Variable (y). **Perform Exploratory Data Analysis**:
4. Generate a Pair Plot for the selected features.
5. Plot a Correlation Heatmap.
6. Identify the feature most strongly correlated with Sales.

Model Planning and Model Building
7. Split the dataset into Training (80%) and Testing (20%) dataset.
8. Build a Linear Regression model and predict Sales for the test dataset.
9. Build a Polynomial Regression model (Degree = 2) and predict Sales for the test dataset. 10. Build a Support Vector Regression (SVR) model with the RBF kernel and predict Sales for the test dataset.

Model Evaluation
11. Evaluate all three models using: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 Score
12. Compare the performance of all models using a single grouped bar chart. 13. Plot Actual vs Predicted Sales for the best-performing model.

Model Interpretation
14. Identify the model with the lowest prediction error.
15. Suggest **three recommendations** to help the company optimize its advertising budget based on the model results.

```python
# Name: Palachuru Nandu Priya
# Roll No: 241801195
#step-1
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```python
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195
#step-2
df = pd.read_csv("advertising.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)

df.head()
```

```
Dataset loaded successfully!
Shape: (200, 4)
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```python
#experiment 3
```

Terminal

```python
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-3
print("Dataset Information:")
df.info()

print("\nNull Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())

# Select features and target
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

print("\nFeatures:", X.columns.tolist())
print("Target: Sales")
```

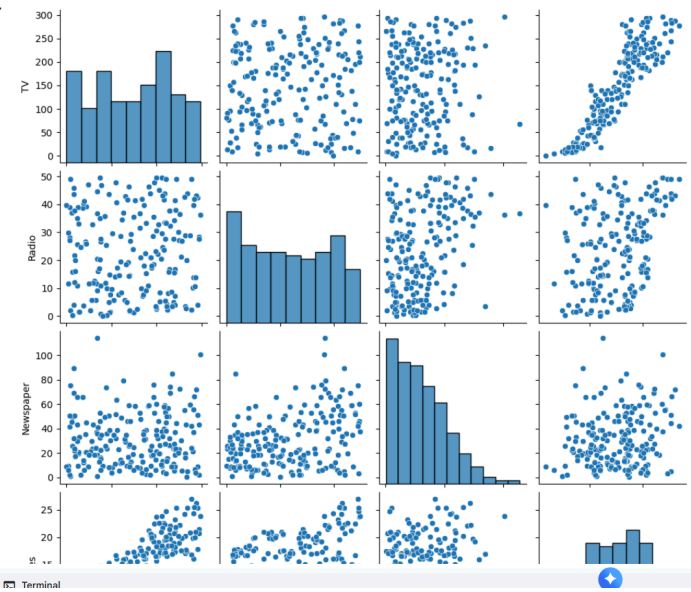
```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB

Null Values:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Statistical Summary:
                TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
```

Terminal

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195
#step-4
sns.pairplot(df[['TV', 'Radio', 'Newspaper', 'Sales']])
plt.show()
```




```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-5
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (160, 3)
Testing data: (40, 3)
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-6
linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Linear Regression Predictions:")
print(y_pred_linear[:10])
```

```
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879 12.56940161
 21.08119452  8.69035045 17.23701254 16.66657475]
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-7
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])

poly_model.fit(X_train, y_train)
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-7
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])

poly_model.fit(X_train, y_train)

y_pred_poly = poly_model.predict(X_test)

print("Polynomial Regression Predictions:")
print(y_pred_poly[:10])
```

```
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435 12.37109504
 22.12872506  8.76904874 17.01492635 16.91480608]
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-8
svr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf'))
])

svr_model.fit(X_train, y_train)

y_pred_svr = svr_model.predict(X_test)

print("SVR Predictions:")
print(y_pred_svr[:10])
```

```
SVR Predictions:
[17.27716452 21.14815556 20.54171345 10.42918914 20.17484466 12.47291227
 21.43267317  9.09307441 17.33861046 16.54760086]
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-9
```

les Terminal

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-9
# Linear Regression
mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)

# Polynomial Regression
mae_poly = mean_absolute_error(y_test, y_pred_poly)
mse_poly = mean_squared_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mse_poly)

# SVR
mae_svr = mean_absolute_error(y_test, y_pred_svr)
mse_svr = mean_squared_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mse_svr)

print("Evaluation completed.")
```

```
Evaluation completed.
```

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-10
results = pd.DataFrame({
    'Model': [
        'Linear Regression',
        'Polynomial Regression',
        'SVR'
    ],
    'MAE': [
        mae_linear,
        mae_poly,
        mae_svr
    ],
    'MSE': [
        mse_linear,
        mse_poly,
```

les Terminal

```
        mse_svr
    ],
    'RMSE': [
        rmse_linear,
        rmse_poly,
        rmse_svr
    ]
})

print("Model Performance Comparison:")
display(results)
```

```
Model Performance Comparison:
```

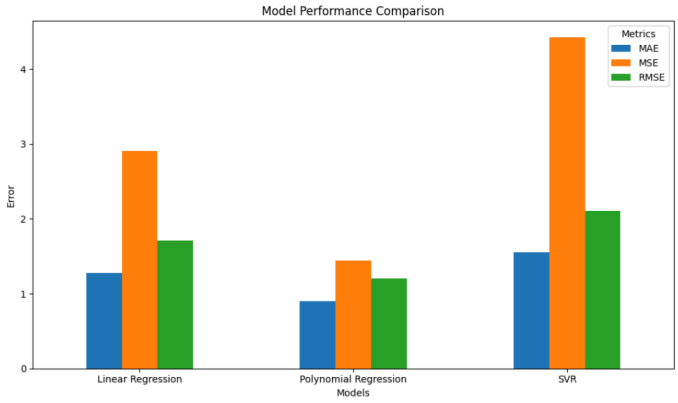
	Model	MAE	MSE	RMSE
0	Linear Regression	1.274826	2.907757	1.705215
1	Polynomial Regression	0.903371	1.442543	1.201059
2	SVR	1.555355	4.421034	2.102626

```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-11
results.set_index('Model')[['MAE', 'MSE', 'RMSE']].plot(
    kind='bar',
    figsize=(10, 6)
)

plt.title('Model Performance Comparison')
plt.xlabel('Models')
plt.ylabel('Error')
plt.xticks(rotation=0)
plt.legend(title='Metrics')
plt.tight_layout()
plt.show()
```



les Terminal




```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-12
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X)

print("Cluster labels assigned successfully.")

print("\nCluster Counts:")
```

les Terminal

```
print(pd.DataFrame(
    kmeans.cluster_centers_,
    columns=['TV', 'Radio', 'Newspaper']
))
```

```
Cluster labels assigned successfully.

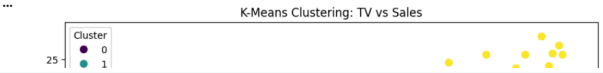
Cluster Counts:
Cluster
0    58
1    65
2    77
Name: count, dtype: int64

Cluster Centers:
           TV      Radio  Newspaper
0  141.796552  20.779310  26.181034
1   45.209231  23.427692  31.064615
2  236.957143  24.997403  33.416883
```

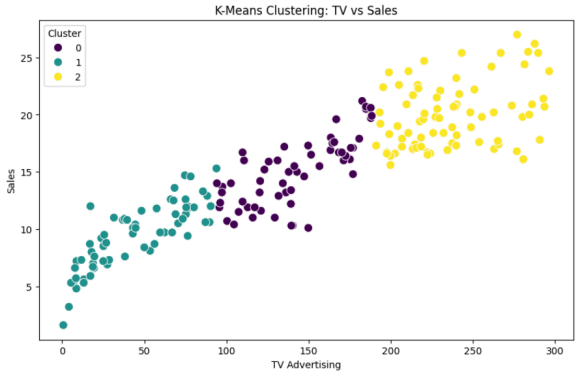
```
#experiment 3
# Name: Palachuru Nandu Priya
# Roll No: 241801195#step-13
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster',
    palette='viridis',
    s=80
)

plt.title('K-Means Clustering: TV vs Sales')
plt.xlabel('TV Advertising')
plt.ylabel('Sales')
plt.legend(title='Cluster')
plt.show()
```



les Terminal



Ex4: Information Retrieval using NLTK

Learning Objective:
The objective of this experiment is to enable students to understand the text preprocessing pipeline and implement a basic information retrieval system using TF-IDF vectorization, Cosine Similarity, NLTK.

Problem Statement:
An e-commerce company receives thousands of customer reviews for its products every day. Customer support executives often need to retrieve reviews that are similar to a user's search query (e.g., "great product with fast shipping" or "poor quality") to quickly understand customer feedback and address common concerns.

As an NLP Engineer, your task is to preprocess customer review text, convert the reviews into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, and develop a simple information retrieval system that returns the most relevant customer reviews for a given search query using Cosine Similarity.

Tasks to Perform:

1. Use Amazon Fine Food Reviews Dataset from Kaggle and load it into a Pandas DataFrame. 2. Install and import the required Python libraries:
3. Import the required NLTK resources:

Clean the dataset

4. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing. **Text Preprocessing**
5. Create a preprocessing function to perform the following operations on each review: Convert text to lowercase, Remove punctuation, numbers, and special characters, Tokenize the text into words, Remove English stopwords, Join the cleaned tokens into a single string.
6. Apply the preprocessing function to all reviews and store the cleaned reviews in a new column named Cleaned_Text.
7. Display the original and cleaned versions of any five reviews.
8. Convert the cleaned review text into TF-IDF feature vectors. Display: Shape of the TF-IDF matrix, Total number of unique terms (vocabulary size).

Information Retrieval

10. Implement a document retrieval function that: Accepts a user query, Applies the same preprocessing steps used for the reviews, Converts the query into a TF-IDF vector, Computes Cosine Similarity between the query and all review vectors, Retrieves the Top 5 most similar reviews, Display Similarity score, Original review, Cleaned review.

Query Evaluation

11. Test the retrieval system using the following sample queries: "great product with fast shipping", "disappointed", "excellent taste", "poor packaging"
12. Compare the retrieved reviews for different queries and comment on their relevance.

ex4 pods

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 1  Load Dataset
from google.colab import files
uploaded = files.upload()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving House Price Prediction Dataset.csv to House Price Prediction Dataset.csv
```

```
df = pd.read_csv("Reviews.csv")
print("First 5 rows:")
print(df.head())
```

```
First 5 rows:
   Id  Area  Bedrooms  Bathrooms  Floors  YearBuilt  Location  Condition  \
0   1  1360         5          4       3       1970  Downtown  Excellent
1   2  4272         5          4       3       1958  Downtown  Excellent
2   3  3592         2          2       3       1938  Downtown       Good
3   4   966         4          2       2       1902  Suburban       Fair
4   5  4926         1          4       2       1975  Downtown       Fair

  Garage   Price
0     No  149919
1     No  424998
2     No  266746
3    Yes  244020
4    Yes  630056
```

ex4 pods

Saving...

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 2
print("\nDataset Information:")
print(df.info())
print("\nStatistical Description:")
print(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Id         2000 non-null   int64
 1   Area       2000 non-null   int64
 2   Bedrooms   2000 non-null   int64
 3   Bathrooms  2000 non-null   int64
 4   Floors     2000 non-null   int64
 5   YearBuilt  2000 non-null   int64
 6   Location   2000 non-null   object
 7   Condition  2000 non-null   object
 8   Garage     2000 non-null   object
 9   Price      2000 non-null   int64
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
None

Statistical Description:
                Id         Area     Bedrooms   Bathrooms       Floors  \
count  2000.000000  2000.000000  2000.000000  2000.00000  2000.000000
mean   1000.500000  2786.209500     3.003500     2.55250     1.993500
std     577.494589  1295.146799     1.424606     1.10899     0.809188
min       1.000000   501.000000     1.000000     1.00000     1.000000
25%     500.750000  1653.000000     2.000000     2.00000     1.000000
50%    1000.500000  2833.000000     3.000000     3.00000     2.000000
75%    1500.250000  3887.500000     4.000000     4.00000     3.000000
max    2000.000000  4999.000000     5.000000     4.00000     3.000000

          YearBuilt          Price
count  2000.000000    2000.000000
mean   1961.446000  537676.855000
std      35.926695  276428.845719
min    1900.000000   50005.000000
25%    1930.000000  300098.000000
50%    1961.000000  539254.000000
75%    1993.000000  780086.000000
max    2023.000000  999656.000000
```

Variables Terminal

ex4 pods

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 3
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Id           0
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 4
X = df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']]
y = df['Price']
print("\nFeatures:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
Features:
   Area  Bedrooms  Bathrooms  Floors  YearBuilt
0  1360         5          4       3       1970
1  4272         5          4       3       1958
2  3592         2          2       3       1938
3   966         4          2       2       1902
4  4926         1          4       2       1975

Target:
0    149919
1    424998
2    266746
3    244020
4    636056
Name: Price, dtype: int64
```

ex4 pods

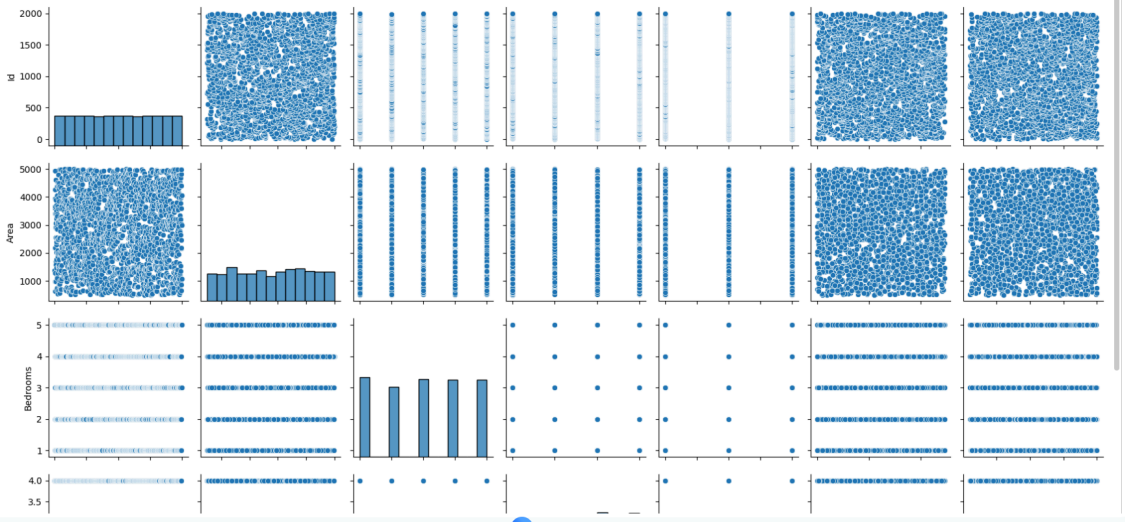
File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 5
sns.pairplot(df)
plt.show()
```



Variables Terminal

ex4 pods

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 6
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 7
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 8
polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 9
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)
```

ex4 pods

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 10
models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}
for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
    print("\nModel:", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
```

```
Model: Linear Regression
MAE : 244587.1999197856
MSE : 78789724000.88458
RMSE: 280695.0729000412

Model: Polynomial Regression
MAE : 245319.05662903108
MSE : 79234323974.83661
RMSE: 281485.9214504992

Model: SVR
MAE : 242647.33691047208
MSE : 77902605845.91304
RMSE: 279110.3829059626
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 11
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
df['Cluster'] = kmeans.labels_
print("\nCluster Labels:")
print(df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Cluster']])
```

ex4 pods

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
Cluster Labels:
      Area  Bedrooms  Bathrooms  Floors  YearBuilt  Cluster
0     1360         5          4       3       1970        0
1     4272         5          4       3       1958        1
2     3592         2          2       3       1938        1
3      966         4          2       2       1902        0
4     4926         1          4       2       1975        1
...    ...       ...        ...     ...        ...      ...
1995  4994         5          4       3       1923        1
1996  3046         5          2       1       2019        2
1997  1062         5          1       2       1903        0
1998  4062         3          1       2       1936        1
1999  2989         5          1       3       1903        2

[2000 rows x 6 columns]
```

```
#palachuru Nandu Priya [241801195]
#ex:4
#step 12
sns.scatterplot(
    data=df,
    x='Area',
    y='Price',
    hue='Cluster'
)
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("House Price Clusters")
plt.show()
```

ex4 pods

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
plt.ylabel("House Price")
plt.title("House Price Clusters")
plt.show()
```



Ex5: Information Extraction using spaCy

Learning Objective:
The objective of this experiment is to enable students to understand and implement information extraction techniques on unstructured customer review text using spaCy.

Problem Statement:
An e-commerce company receives thousands of customer reviews containing valuable information about products, brands, organizations, locations, quantities, and other entities. Manually identifying and organizing this information from large volumes of reviews is time consuming and inefficient.

As an NLP Engineer, your task is to process customer reviews using the spaCy Natural Language Processing library and develop an information extraction system that identifies and extracts meaningful linguistic information and named entities from the reviews. The extracted information should be organized into a structured format that can be further used for customer feedback analysis.

Tasks to Perform:

1. Reuse the Amazon Fine Food Reviews Dataset used in the previous experiment and load it into a Pandas DataFrame. Install and import the required libraries.

Clean the dataset

2. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing. **Text Preprocessing and Part-of-speech (POS) tags**

3. Apply the spaCy pipeline to the review text. For any five reviews, display: Original review, Tokens, Lemmas, Part-of-speech (POS) tags.
4. Identify and display different POS categories such as: Nouns, Verbs, Adjectives, Adverbs. 5. Count and display the frequency of the major POS categories present in the reviews. **Named Entity Recognition (NER)**

6. Apply Named Entity Recognition (NER) using spaCy to identify entities in the reviews. Extract and display entities such as: PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT, where recognized by the selected spaCy model. Display the extracted information in a structured DataFrame.

Information Extraction

7. Create a function that accepts a customer review as input and returns: Tokens, Lemmas, POS tags, Named entities, Entity labels.
8. Test the function using at least three new customer reviews.
9. Display the extracted information in a structured format.

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[9]
```
#palachuru Nandu Priya [241801195]

import pandas as pd
import spacy
from collections import Counter
```

[10]
```
#palachuru Nandu Priya [241801195]

%pip install pandas spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.4)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.5.4)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.2)
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.5.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.13.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (26.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic!<3.0.0,>=2.0.0->spacy) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic!<3.0.0,>=2.0.0->spacy) (2.46.5)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.13/dist-packages (from pydantic!<3.0.0,>=2.0.0->spacy) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic!<3.0.0,>=2.0.0->spacy) (0.4.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.7.22)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.5)
```

Variables Terminal Python 3

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 22.0 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

[12]
```
#palachuru Nandu Priya [241801195]

df = pd.read_csv("Amazon_Fine_Food_Reviews_Practice.csv")

print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe(include="all"))
```

```
   ReviewID                                               Text  Rating  \
0         1  I love this organic green tea from Amazon. The...       4
1         2  Nature's Bakery makes excellent fig bars; I bo...       5
2         3  The headphones from Sony are comfortable, but ...       1
3         4  Dr. Emily Carter recommended this vitamin supp...       5
4         5  This coffee maker by Philips is worth $89 and ...       1

  ProductCategory Reviewer  ReviewDate
0          Beauty    Aarav  2025-01-01
1     Electronics    Meera  2025-01-02
2     Electronics    Priya  2025-01-03
3           Books    Meera  2025-01-04
4           Books    Rahul  2025-01-05
     ReviewID                                               Text  Rating  \
115       116  The Philips mug is useful. I bought 3 units fo...       5
116       117  The Sony backpack is affordable. I bought 4 un...       2
117       118  The Sony charger is useful. I bought 4 units f...       2
118       119  The Target tea is useful. I bought 3 units for...       5
119       120  The Cetaphil water bottle is useful. I bought ...       5

    ProductCategory Reviewer  ReviewDate
115            Food    Priya  2025-04-26
116          Beauty    Aarav  2025-04-27
117           Books    Aarav  2025-04-28
118            Home    Rahul  2025-04-29
119           Books    Priya  2025-04-30
Shape: (120, 6)
ReviewID            int64
Text               object
Rating              int64
```

Variables Terminal Python 3

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
[13] #palachuru Nandu Priya [241801195]

df = df[["Text"]].dropna()
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df = df.drop_duplicates(subset="Text")
df = df.head(10000).reset_index(drop=True)

print("Cleaned shape:", df.shape)
print(df.head())
```

```
Cleaned shape: (120, 1)
                                                Text
0  I love this organic green tea from Amazon. The...
1  Nature's Bakery makes excellent fig bars; I bo...
2  The headphones from Sony are comfortable, but ...
3  Dr. Emily Carter recommended this vitamin supp...
4  This coffee maker by Philips is worth $89 and ...
```

```
[14] #palachuru Nandu Priya [241801195]

nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully")
```

```
spaCy model loaded successfully
```

```
[15] #palachuru Nandu Priya [241801195]

for review in df["Text"].head(5):
    doc = nlp(review)

    print("\nReview:", review)
    print("Tokens:", [token.text for token in doc])
    print("Lemmas:", [token.lemma_ for token in doc])
    print("POS Tags:", [(token.text, token.pos_) for token in doc])
```

```
Review: I love this organic green tea from Amazon. The package arrived on March 5, 2024, and cost $12.99.
Tokens: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'The', 'package', 'arrived', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
Lemmas: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'the', 'package', 'arrive', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
POS Tags: [('I', 'PRON'), ('love', 'VERB'), ('this', 'DET'), ('organic', 'ADJ'), ('green', 'ADJ'), ('tea', 'NOUN'), ('from', 'ADP'), ('Amazon', 'PROPN'), ('.', 'PUNCT'), ('The', 'DET'), ('package', 'NOUN'), ('arrived', 'VERB

Review: Nature's Bakery makes excellent fig bars; I bought 3 boxes yesterday.
Tokens: ['Nature', "'s", 'Bakery', 'makes', 'excellent', 'fig', 'bars', ';', 'I', 'bought', '3', 'boxes', 'yesterday', '.']
Lemmas: ['nature', "'s", 'Bakery', 'make', 'excellent', 'fig', 'bar', ';', 'I', 'buy', '3', 'box', 'yesterday', '.']
POS Tags: [('Nature', 'NOUN'), ("'s", 'PART'), ('Bakery', 'PROPN'), ('makes', 'VERB'), ('excellent', 'ADJ'), ('fig', 'NOUN'), ('bars', 'NOUN'), (';', 'PUNCT'), ('I', 'PRON'), ('bought', 'VERB'), ('3', 'NUM'), ('boxes', 'NOUN

Review: The headphones from Sony are comfortable, but the battery lasted only 4 hours.
Tokens: ['The', 'headphones', 'from', 'Sony', 'are', 'comfortable', ',', 'but', 'the', 'battery', 'lasted', 'only', '4', 'hours', '.']
Lemmas: ['the', 'headphone', 'from', 'Sony', 'be', 'comfortable', ',', 'but', 'the', 'battery', 'last', 'only', '4', 'hour', '.']
POS Tags: [('The', 'DET'), ('headphones', 'NOUN'), ('from', 'ADP'), ('Sony', 'PROPN'), ('are', 'AUX'), ('comfortable', 'ADJ'), (',', 'PUNCT'), ('but', 'CCONJ'), ('the', 'DET'), ('battery', 'NOUN'), ('lasted', 'VERB'), ('only

Review: Dr. Emily Carter recommended this vitamin supplement in 2023.
Tokens: ['Dr.', 'Emily', 'Carter', 'recommended', 'this', 'vitamin', 'supplement', 'in', '2023', '.']
Lemmas: ['Dr.', 'Emily', 'Carter', 'recommend', 'this', 'vitamin', 'supplement', 'in', '2023', '.']
POS Tags: [('Dr.', 'PROPN'), ('Emily', 'PROPN'), ('Carter', 'PROPN'), ('recommended', 'VERB'), ('this', 'DET'), ('vitamin', 'NOUN'), ('supplement', 'NOUN'), ('in', 'ADP'), ('2023', 'NUM'), ('.', 'PUNCT')]

Review: This coffee maker by Philips is worth $89 and arrived in two days.
Tokens: ['This', 'coffee', 'maker', 'by', 'Philips', 'is', 'worth', '$', '89', 'and', 'arrived', 'in', 'two', 'days', '.']
Lemmas: ['this', 'coffee', 'maker', 'by', 'Philips', 'be', 'worth', '$', '89', 'and', 'arrive', 'in', 'two', 'day', '.']
POS Tags: [('This', 'DET'), ('coffee', 'NOUN'), ('maker', 'NOUN'), ('by', 'ADP'), ('Philips', 'PROPN'), ('is', 'AUX'), ('worth', 'ADJ'), ('$', 'SYM'), ('89', 'NUM'), ('and', 'CCONJ'), ('arrived', 'VERB'), ('in', 'ADP'), ('tw
```

```
[16] #palachuru Nandu Priya [241801195]

counts = Counter()

for review in df["Text"]:
    doc = nlp(review)
    counts.update(token.pos_ for token in doc)

doc = nlp(df["Text"][0])

print("Nouns:", [t.text for t in doc if t.pos_ in ["NOUN", "PROPN"]])
print("Verbs:", [t.text for t in doc if t.pos_ == "VERB"])
print("Adjectives:", [t.text for t in doc if t.pos_ == "ADJ"])
print("Adverbs:", [t.text for t in doc if t.pos_ == "ADV"])

for pos in ["NOUN", "PROPN", "VERB", "AUX", "ADJ", "ADV"]:
    print(pos, ":", counts[pos])
```

```
Nouns: ['tea', 'Amazon', 'package', 'March']
Verbs: ['love', 'arrived', 'cost']
Adjectives: ['organic', 'green']
Adverbs: []
NOUN : 640
```

Variables Terminal

Python 3

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[17]

```
#palachuru Nandu Priya [241801195]


entities = []

for i, review in enumerate(df["Text"], start=1):
    doc = nlp(review)

    for ent in doc.ents:
        entities.append({
            "ReviewID": i,
            "Review": review,
            "Entity": ent.text,
            "Entity Label": ent.label_
        })

entities_df = pd.DataFrame(entities)

print(entities_df.head(20))
```

```
    ReviewID                                             Review  \
0          1  I love this organic green tea from Amazon. The...
1          1  I love this organic green tea from Amazon. The...
2          1  I love this organic green tea from Amazon. The...
3          2  Nature's Bakery makes excellent fig bars; I bo...
4          2  Nature's Bakery makes excellent fig bars; I bo...
5          2  Nature's Bakery makes excellent fig bars; I bo...
6          3  The headphones from Sony are comfortable, but ...
7          3  The headphones from Sony are comfortable, but ...
8          4  Dr. Emily Carter recommended this vitamin supp...
9          4  Dr. Emily Carter recommended this vitamin supp...
10         5  This coffee maker by Philips is worth $89 and ...
11         5  This coffee maker by Philips is worth $89 and ...
12         5  This coffee maker by Philips is worth $89 and ...
13         6  The chocolate tastes delicious and the 500 gra...
14         7  Apple customer service replaced my damaged dev...
15         7  Apple customer service replaced my damaged dev...
16         8  I ordered two bottles of olive oil from Califo...
17         8  I ordered two bottles of olive oil from Califo...
18         9  The book by John Smith explains machine learni...
19        10  These running shoes cost 75 dollars and were d...

           Entity Entity Label
0          Amazon          ORG
1  March 5, 2024         DATE
2           12.99        MONEY
3             fig          ORG
4               3     CARDINAL
5       yesterday         DATE
6           Sony          ORG
```

Variables Terminal

Python 3

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[18]

```
#palachuru Nandu Priya [241801195]


def extract_information(review):
    doc = nlp(review)

    return {
        "Tokens": [t.text for t in doc],
        "Lemmas": [t.lemma_ for t in doc],
        "POS Tags": [t.pos_ for t in doc],
        "Named Entities": [e.text for e in doc.ents],
        "Entity Labels": [e.label_ for e in doc.ents]
    }

reviews = [
    "Ravi bought a Sony speaker for $45 in Chennai.",
    "Priya purchased two bottles of shampoo yesterday.",
    "The Samsung phone arrived on January 12, 2025."
]

for review in reviews:
    print("\nReview:", review)
    print(extract_information(review))
```

```
Review: Ravi bought a Sony speaker for $45 in Chennai.
{'Tokens': ['Ravi', 'bought', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'Lemmas': ['ravi', 'buy', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'POS Tags': ['NOUN', 'VERB', 'DET', 'P

Review: Priya purchased two bottles of shampoo yesterday.
{'Tokens': ['Priya', 'purchased', 'two', 'bottles', 'of', 'shampoo', 'yesterday', '.'], 'Lemmas': ['Priya', 'purchase', 'two', 'bottle', 'of', 'shampoo', 'yesterday', '.'], 'POS Tags': ['PROPN', 'VERB', 'NUM', 'NOUN', 'ADP',

Review: The Samsung phone arrived on January 12, 2025.
{'Tokens': ['The', 'Samsung', 'phone', 'arrived', 'on', 'January', '12', ',', '2025', '.'], 'Lemmas': ['the', 'Samsung', 'phone', 'arrive', 'on', 'January', '12', ',', '2025', '.'], 'POS Tags': ['DET', 'PROPN', 'NOUN', 'VERB
```

pods-Spacy-5th.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[19]

```
#palachuru Nandu Priya [241801195]


entities_df.to_csv("ex5_extracted_entities.csv", index=False)

results = []

for i, review in enumerate(df["Text"], start=1):
    info = extract_information(review)

    results.append({
        "ReviewID": i,
        "Review": review,
        "Tokens": ", ".join(info["Tokens"]),
        "Lemmas": ", ".join(info["Lemmas"]),
        "POS Tags": ", ".join(info["POS Tags"]),
        "Named Entities": ", ".join(info["Named Entities"]),
        "Entity Labels": ", ".join(info["Entity Labels"])
    })

result_df = pd.DataFrame(results)
result_df.to_csv("ex5_review_information.csv", index=False)

print("Files saved successfully!")
print(result_df.head())
```

```
Files saved successfully!
   ReviewID                                             Review  \
0         1  I love this organic green tea from Amazon. The...
1         2  Nature's Bakery makes excellent fig bars; I bo...
2         3  The headphones from Sony are comfortable, but ...
3         4  Dr. Emily Carter recommended this vitamin supp...
4         5  This coffee maker by Philips is worth $89 and ...

                                              Tokens  \
0  I, love, this, organic, green, tea, from, Amaz...
1  Nature, 's, Bakery, makes, excellent, fig, bar...
2  The, headphones, from, Sony, are, comfortable,...
3  Dr., Emily, Carter, recommended, this, vitamin...
4  This, coffee, maker, by, Philips, is, worth, $...

                                              Lemmas  \
0  I, love, this, organic, green, tea, from, Amaz...
1  nature, 's, Bakery, make, excellent, fig, bar,...
2  the, headphone, from, Sony, be, comfortable, ,...
3  Dr., Emily, Carter, recommend, this, vitamin, ...
4  this, coffee, maker, by, Philips, be, worth, $...

                                            POS Tags  \
```

Variables Terminal

Python 3

Ex6a: Exploratory Data Analysis

Learning Objective:
The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA)on a real-world Netflix titles dataset using Python.

Problem Statement:
Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. Netflix wants to understand the characteristics and growth of its content library and identify important patterns in its movies and TV shows.

As a Data Scientist, perform an Exploratory Data Analysis on the Netflix Titles dataset. Analyse the content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions. Use suitable statistical techniques and visualisations to identify patterns and generate meaningful insights from the data.

Tasks to Perform:

1. Download Netflix_titles dataset from kaggle and convert it to dataframe. 2. Explore the Dataset
3. **Content Distribution Analysis. (Plot graphs wherever needed)**
    a. How many Movies vs TV Shows are in the dataset?
    b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?
    c. Which countries have the most Netflix content?
4. **Time-Based Trend Analysis (Plot graphs wherever needed)**
    a. In which years were the most shows/movies released?
    b. In which years were titles added to Netflix?
    c. How has Netflix's content growth trend changed over time?
5. **Duration & Content Type Analysis (Plot graphs wherever needed)** a. What is the distribution of movie durations (in minutes)?
    b. How many TV shows exist per number of seasons?
6. **Genre & Categories Analysis (Plot graphs wherever needed)**
    a. What are the top 10 most common genres on Netflix?
    b. Which directors have the most titles on Netflix?
    c. Which actors appear most frequently across Netflix shows/movies?
7. **Text & Descriptions**
    a. What are the most common words in the description field? (word cloud) 8. In which months of the year does Netflix add the most content?
9. Which countries produce more movies vs TV shows**?**
10. What are the most common ratings for Movies vs TV Shows?
11. What are the oldest and newest titles available on Netflix?
12. Which genres frequently co-occur together**?** (Genre correlation heatmap)

```
#Experiment 6a - Exploratory Data Analysis
#Palachuru Nandu Priya [241801195]

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from collections import Counter
from wordcloud import WordCloud

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
```

```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-1
from google.colab import files

uploaded = files.upload()
df = pd.read_csv("netflix_titles.csv")

print("Dataset Shape:", df.shape)
df.head()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving netflix_titles.csv to netflix_titles.csv
Dataset Shape: (8807, 12)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	des
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a C
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a pow
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet tal
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers kno

```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-2
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())
df.describe(include="all").T
```

```
Column Names:
['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_in', 'description']

Data Types:
show_id         object
type            object
title           object
director        object
cast            object
country         object
date_added      object
release_year     int64
rating          object
duration        object
listed_in       object
description     object
dtype: object

Missing Values:
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
```

```python
#Experiment 8a
#Palachuru Nandu Priya [241801195]
#step-3
content_count = df["type"].value_counts()

print(content_count)

plt.figure(figsize=(7,5))
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
rating_count = df["rating"].value_counts()

print(rating_count)

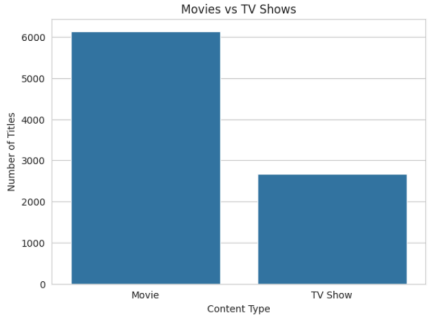
plt.figure(figsize=(10,6))
sns.barplot(x=rating_count.values, y=rating_count.index)
plt.title("Netflix Content Ratings")
plt.xlabel("Number of Titles")
plt.ylabel("Rating")
plt.show()
countries = (
    df["country"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(countries.head(10))

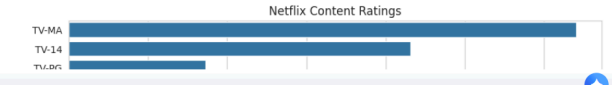
plt.figure(figsize=(10,6))
sns.barplot(x=countries.head(10).values,
            y=countries.head(10).index)

plt.title("Top 10 Countries Producing Netflix Content")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
```

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```




```
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
NC-17          3
UR             3
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64
```



```python
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-4
release_years = df["release_year"].value_counts().sort_index()

plt.figure(figsize=(12,6))
plt.plot(release_years.index, release_years.values)
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()

print("Top Release Years:")
print(df["release_year"].value_counts().head(10))
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")

added_year = df["date_added"].dt.year.value_counts().sort_index()

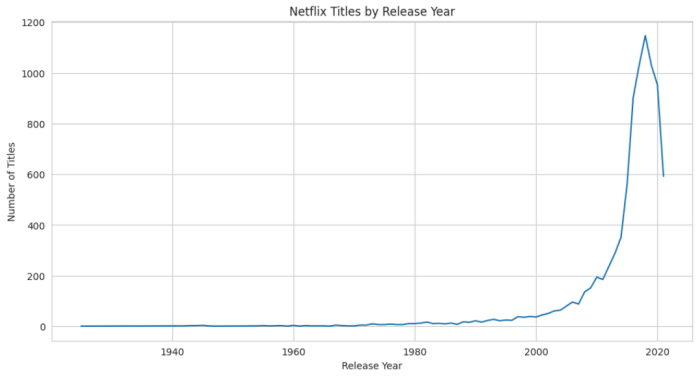
plt.figure(figsize=(10,6))
plt.plot(added_year.index, added_year.values, marker="o")
plt.title("Netflix Titles Added by Year")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.show()
growth = df.dropna(subset=["date_added"]).copy()

growth["year_added"] = growth["date_added"].dt.year

growth_count = growth.groupby("year_added").size().sort_index()
cumulative_growth = growth_count.cumsum()

plt.figure(figsize=(10,6))
plt.plot(cumulative_growth.index,
         cumulative_growth.values,
         marker="o")

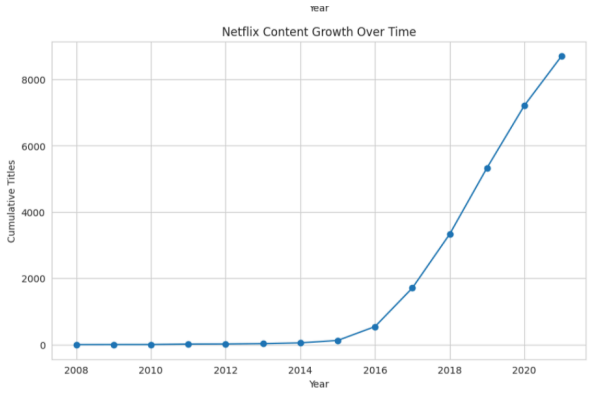
plt.title("Netflix Content Growth Over Time")
plt.xlabel("Year")
plt.ylabel("Cumulative Titles")
```




```
Top Release Years:
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
2015     560
2014     352
2013     288
2012     237
Name: count, dtype: int64
```

```
Name: count, dtype: int64
```






```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-5
movies = df[df["type"] == "Movie"].copy()

movies["duration_min"] = (
    movies["duration"]
    .str.extract(r"(\d+)")
    .astype(float)
)

print(movies["duration_min"].describe())

plt.figure(figsize=(10,6))
sns.histplot(movies["duration_min"].dropna(), bins=30, kde=True)

plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (minutes)")
plt.ylabel("Number of Movies")
plt.show()
tv = df[df["type"] == "TV Show"].copy()

tv["seasons"] = (
    tv["duration"]
    .str.extract(r"(\d+)")
    .astype(float)
)

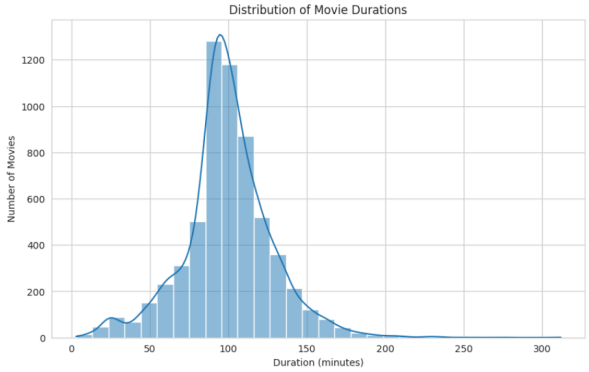
season_count = tv["seasons"].value_counts().sort_index()

print(season_count)

plt.figure(figsize=(10,6))
sns.barplot(x=season_count.index.astype(str),
            y=season_count.values)

plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

```
count    6128.000000
mean       99.577187
std        28.290593
min         3.000000
25%        87.000000
50%        98.000000
75%       114.000000
max       312.000000
Name: duration_min, dtype: float64
```




```
seasons
1.0     1793
2.0      425
3.0      199
4.0       95
5.0       65
6.0       33
7.0       23
8.0       17
9.0        9
10.0       7
11.0       2
12.0       2
13.0       3
15.0       2
17.0       1
```

```
#Palachuru Nandu Priya [241801195]
#step-6
genres = (
    df["listed_in"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(genres.head(10))

plt.figure(figsize=(10,6))
sns.barplot(x=genres.head(10).values,
            y=genres.head(10).index)

plt.title("Top 10 Most Common Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.show()
directors = (
    df["director"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(directors.head(10))

plt.figure(figsize=(10,6))
sns.barplot(x=directors.head(10).values,
            y=directors.head(10).index)

plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.show()
actors = (
```

```
plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.show()
actors = (
    df["cast"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(actors.head(10))

plt.figure(figsize=(10,6))
sns.barplot(x=actors.head(10).values,
            y=actors.head(10).index)

plt.title("Top 10 Most Frequent Actors")
plt.xlabel("Number of Titles")
plt.ylabel("Actor")
plt.show()
```

```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```

```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-7
text = " ".join(df["description"].dropna().astype(str))

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    stopwords=set(["the", "and", "of", "to", "a", "in", "with", "on"])
).generate(text)

plt.figure(figsize=(15,7))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Most Common Words in Netflix Descriptions")
plt.show()
```



```python
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-8
month_count = (
    df["date_added"]
    .dt.month_name()
    .value_counts()
)

month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

month_count = month_count.reindex(month_order)

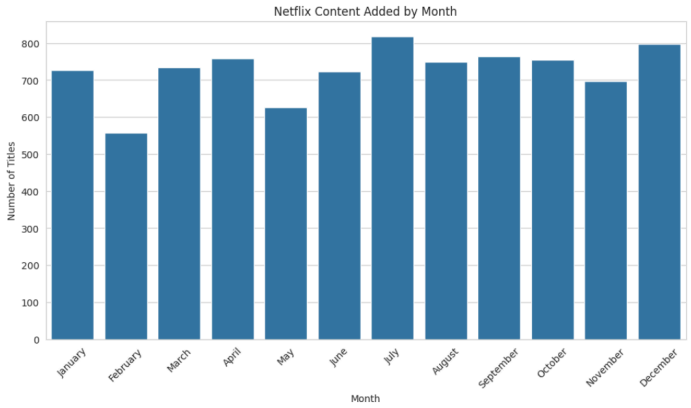
print(month_count)

plt.figure(figsize=(12,6))
sns.barplot(x=month_count.index,
            y=month_count.values)

plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
date_added
January      727
February     557
March        734
April        759
May          626
June         724
July         819
August       749
September    765
October      755
November     697
December     797
Name: count, dtype: int64
```

```
December     739
Name: count, dtype: int64
```




```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-9
country_type = df.dropna(subset=["country"]).copy()

country_type["country"] = country_type["country"].str.split(", ")
country_type = country_type.explode("country")

top_countries = (
    country_type["country"]
    .value_counts()
    .head(10)
    .index
```

```
country_comparison = pd.crosstab(
    country_type["country"],
    country_type["type"]
).loc[top_countries]

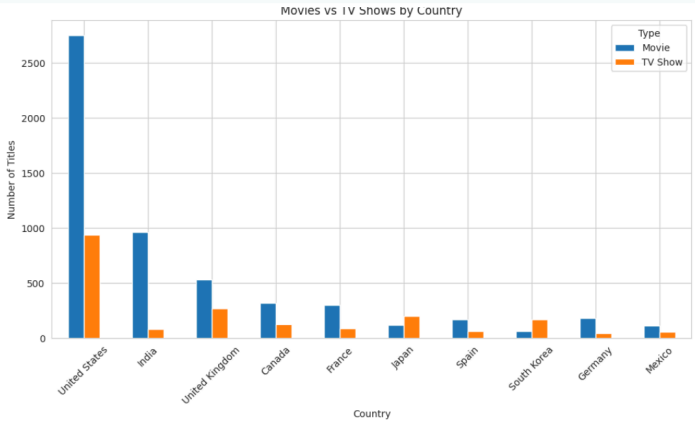
print(country_comparison)

country_comparison.plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.legend(title="Type")
plt.show()
```

```
type            Movie  TV Show
country
United States    2751      938
India             962       84
United Kingdom    532      272
Canada            319      126
France            303       90
Japan             119      199
Spain             171       61
South Korea        61      170
Germany           182       44
Mexico            111       58
```





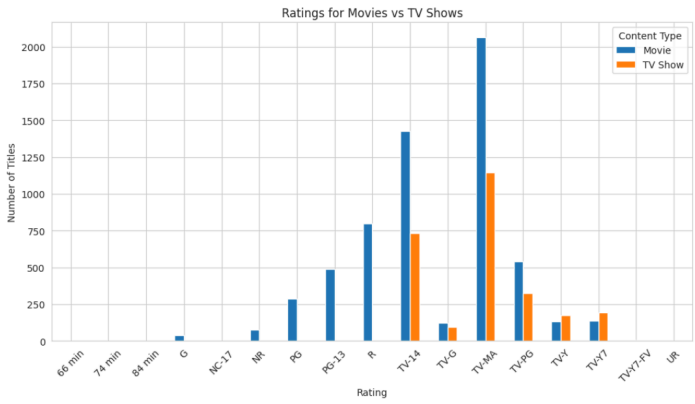

```
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-10
rating_type = pd.crosstab(
    df["rating"],
    df["type"]
)

print(rating_type)

rating_type.plot(
    kind="bar",
    figsize=(12,6)
)
```

```
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.legend(title="Content Type")
plt.show()
```

```
type       Movie  TV Show
rating
66 min         1        0
74 min         1        0
84 min         1        0
G             41        0
NC-17          3        0
NR            75        5
PG           287        0
PG-13        490        0
R            797        2
TV-14       1427      733
TV-G         126       94
TV-MA       2062     1145
TV-PG        540      323
TV-Y         131      176
TV-Y7        139      195
TV-Y7-FV       5        1
UR             3        0
```



```python
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-11
oldest = df.loc[df["release_year"].idxmin()]
newest = df.loc[df["release_year"].idxmax()]

print("OLDEST TITLE")
print("Title:", oldest["title"])
print("Release Year:", oldest["release_year"])
print("Type:", oldest["type"])

print("\nNEWEST TITLE")
print("Title:", newest["title"])
print("Release Year:", newest["release_year"])
print("Type:", newest["type"])
```

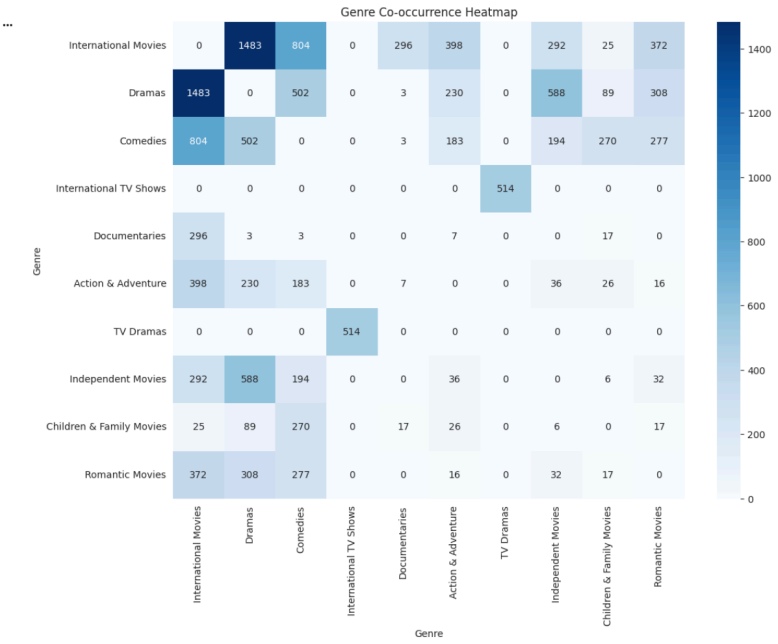
```
OLDEST TITLE
Title: Pioneers: First Women Filmmakers*
Release Year: 1925
Type: TV Show

NEWEST TITLE
Title: Blood & Water
Release Year: 2021
Type: TV Show
```

```python
#Experiment 6a
#Palachuru Nandu Priya [241801195]
#step-12
genre_lists = (
    df["listed_in"]
    .dropna()
    .str.split(", ")
)

top_genres = genres.head(10).index.tolist()

genre_matrix = pd.DataFrame(
    0,
    index=top_genres,
    columns=top_genres
)
```




```
#Experiment 6a
#Palachuru Nandu Priya [241801195]-
#step-13
print("========== NETFLIX EDA SUMMARY ==========\n")

print("Total Titles:", len(df))

print("\nContent Distribution:")
print(df["type"].value_counts())

print("\nTop 5 Countries:")
print(countries.head(5))

print("\nTop 5 Genres:")
print(genres.head(5))

print("\nTop 5 Directors:")
print(directors.head(5))

print("\nTop 5 Actors:")
print(actors.head(5))

print("\nMost Common Ratings:")
print(df["rating"].value_counts().head(5))

print("\nOldest Release Year:", df["release_year"].min())
print("Newest Release Year:", df["release_year"].max())

print("\nMonth with Most Additions:",
      month_count.idxmax())

print("========================================")
```

```
========== NETFLIX EDA SUMMARY ==========

Total Titles: 8807

Content Distribution:
type
Movie      6131
TV Show    2676
```

Ex6b: Time Series Data Analysis and Forecast

Learning Objective:
The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement:
An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform:

1. Download airline-passenges dataset from https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv 2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test. 6. If the series is non-stationary, convert it to stationary by differencing the series. 7. Re-run ADF on the differenced series and visualize the new dataframe. 8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
9. Split the dataset for training and testing on original series.
10. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
11. Perform forecasts on the test dataset.
12. Compute metrics MSE, MAE, RMSE on the predictions.
13. Plot train, test, and predictions in a single plot.

pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
# Step 1 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

from google.colab import files
import pandas as pd
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

```
Choose Files No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving airline-passengers.csv to airline-passengers.csv
```

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

+ Code + Text

```
# Step 2 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

df = pd.read_csv(
    list(uploaded.keys())[0],
    parse_dates=['Month'],
    index_col='Month'
)
df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```
# Step 3 - Name:Palachuru Nandu Priya[241801195] - Ex:5B
```

Variables Terminal

pods 5b.ipynb Saving...

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

Connect

```
# Step 3 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

```
(144, 1)
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
Passengers    0
dtype: int64
```

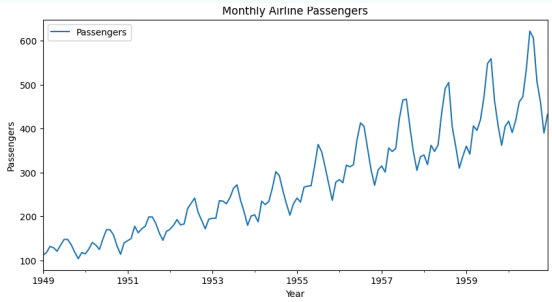
```
# Step 4 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

import matplotlib.pyplot as plt
df.plot(figsize=(10,5))
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.show()
```

pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect




```
# Step 5 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

from statsmodels.tsa.stattools import adfuller
result = adfuller(df['Passengers'])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
/tmp/ipykernel_1984/2905564206.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavi
  result = adfuller(df['Passengers'])
```

pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

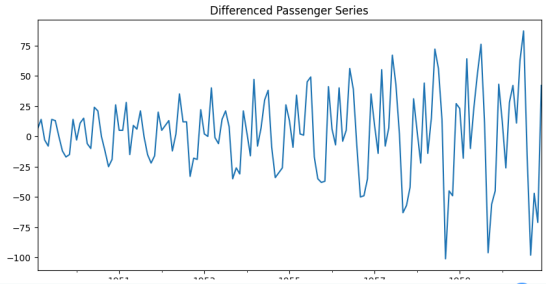
```
# Step 6 -Name:Palachuru Nandu Priya[241801195] - Ex:5B

diff = df['Passengers'].diff().dropna()
print(diff.head())
```

```
Month
1949-02-01    6.0
1949-03-01   14.0
1949-04-01   -3.0
1949-05-01   -8.0
1949-06-01   14.0
Name: Passengers, dtype: float64
```

```
# Step 7 - Name:Palachuru Nandu Priya[241801195]  - Ex:5B
result = adfuller(diff)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
diff.plot(figsize=(10,5))
plt.title("Differenced Passenger Series")
plt.show()
```

```
/tmp/ipykernel_1984/4064224130.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavi
  result = adfuller(diff)
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
```



Variables Terminal

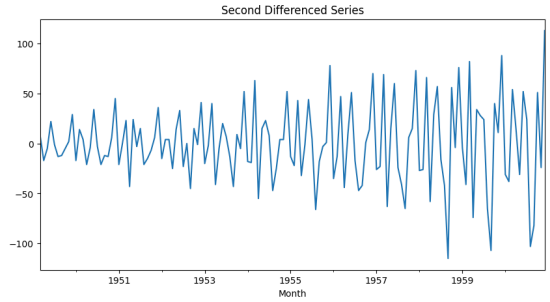
pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
# Step 8 - Name:Palachuru Nandu Priya[241801195]  - Ex:5B
diff2 = diff.diff().dropna()
result = adfuller(diff2)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
diff2.plot(figsize=(10,5))
plt.title("Second Differenced Series")
plt.show()
```

```
/tmp/ipykernel_1984/2335684628.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavi
  result = adfuller(diff2)
ADF Statistic: -16.384231542468513
p-value: 2.7328918500142026e-29
```



pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
# Step 9 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

train = df['Passengers'][:-12]
test = df['Passengers'][-12:]
print("Training:", len(train))
print("Testing:", len(test))
```

```
Training: 132
Testing: 12
```

```
# Step 10 - Name:Palachuru Nandu Priya[241801195]  - Ex:5B
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(1,1,1))
model_fit = model.fit()
print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  132
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -625.518
Date:                Tue, 06 Oct 2026   AIC                           1257.035
Time:                        03:24:34   BIC                           1265.661
Sample:                  01-01-1949   HQIC                          1260.540
                       - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5436      0.100     -5.443      0.000      -0.739      -0.348
ma.L1          0.9272      0.051     18.336      0.000       0.828       1.026
sigma2       816.5398     95.147      8.582      0.000     630.056    1003.024
===================================================================================
Ljung-Box (L1) (Q):                   0.31   Jarque-Bera (JB):                 2.51
Prob(Q):                              0.57   Prob(JB):                         0.29
Heteroskedasticity (H):               6.76   Skew:                            -0.27
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.42
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
# Step 11 - Name:Palachuru Nandu Priya[241801195]- Ex:5B

forecast = model_fit.forecast(steps=len(test))
print(forecast)
```

```
1960-01-01    433.451927
1960-02-01    417.984221
1960-03-01    426.393139
1960-04-01    421.821685
1960-05-01    424.306927
1960-06-01    422.955841
1960-07-01    423.690350
1960-08-01    423.291039
1960-09-01    423.508122
1960-10-01    423.390106
1960-11-01    423.454265
1960-12-01    423.419385
Freq: MS, Name: predicted_mean, dtype: float64
```

```
# Step 12 - Name:Palachuru Nandu Priya[241801195] - Ex:5B

from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)
print("MSE:", mse)
print("MAE:", mae)
print("RMSE:", rmse)
```

```
MSE: 8322.701142131522
MAE: 66.24167467277367
RMSE: 91.228839421585
```

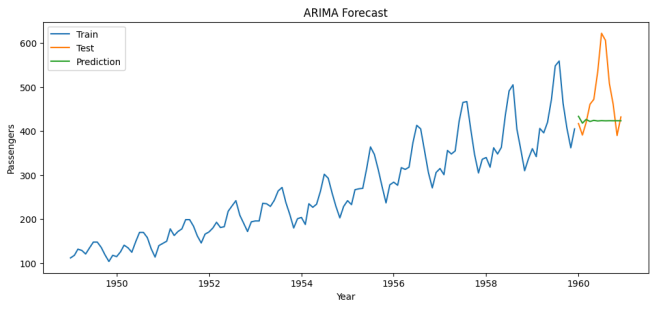
pods 5b.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
# Step 13 - Name:Palachuru Nandu Priya[241801195]- Ex:5B

plt.figure(figsize=(12,5))
plt.plot(train, label='Train')
plt.plot(test, label='Test')
plt.plot(test.index, forecast, label='Prediction')
plt.title("ARIMA Forecast")
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```



Ex7: Model Planning and Building - Clustering

Learning Objective:
The objective of this experiment is to enable students to understand and implement the K Means clustering algorithm for customer segmentation.

Problem Statement:
A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. By identifying customers with similar purchasing behavior, the company can improve customer satisfaction, optimize promotional offers, and increase overall sales.

Your task is to analyze the customer dataset, apply the K-Means Clustering algorithm to identify customer segments, visualize the clusters, and recommend suitable marketing strategies for each segment.

Tasks to Perform:

1. Use Mall Customer Segmentation Dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.
3. Select the important features for clustering: Annual Income (k$), Spending Score (1–100).
4. Standardize the selected features using StandardScaler for clustering. 5. Visualize the selected features using: Scatter Plot, Pair Plot.

Model Planning and Building

6. Determine the optimal number of clusters using the Elbow Method. Plot the Elbow Curve and identify the optimal value of K.
7. Train the K-Means Clustering model using the optimal number of clusters. Assign cluster labels to each customer, add as a new column in the dataset and save as a file. **Cluster Visualization**
8. Visualize the customer clusters using a scatter plot: X-axis: Annual Income, Y-axis: Spending Score, Different colors for each cluster, Highlight cluster centroids, Display the coordinates of the cluster centroids.

Cluster Analysis

9. Calculate the number of customers in each cluster.
10. Compute the average Annual Income and Spending Score for each cluster.

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from google.colab import files
```

[2]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

uploaded = files.upload()
df = pd.read_csv('Mall_Customers (2).csv')

print("First Five Records:")
display(df.head())

print("Last Five Records:")
display(df.tail())
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Mall_Customers (2).csv to Mall_Customers (2).csv
First Five Records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Female	28	15	35
1	2	Female	62	17	82
2	3	Male	60	18	70
3	4	Male	70	19	1
4	5	Male	52	19	10

```
Last Five Records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	63	134	2
196	197	Female	57	135	69
197	198	Female	57	135	83
198	199	Female	38	137	20

Variables Terminal Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

print("Dataset Shape:", df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:", df.duplicated().sum())

print("\nSummary Statistics:")
display(df.describe(include='all'))
```

```
Dataset Shape: (200, 5)

Data Types:
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object

Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Duplicate Records: 0

Summary Statistics:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200	200.000000	200.000000	200.000000
unique	NaN	2	NaN	NaN	NaN
top	NaN	Female	NaN	NaN	NaN
freq	NaN	102	NaN	NaN	NaN
mean	100.500000	NaN	45.635000	77.110000	51.500000
std	57.879185	NaN	15.383597	33.830763	28.970788

Variables Terminal Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[4]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

print("Selected Features:")
display(X.head())

print("\nFeature Shape:", X.shape)
```

```
Selected Features:
```

	Annual Income (k$)	Spending Score (1-100)
0	15	35
1	17	82
2	18	70
3	19	1
4	19	10

```
Feature Shape: (200, 2)
```

[5]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (Standardized)',
             'Spending Score (Standardized)']
)

print("Standardized Features:")
display(X_scaled_df.head())
```

```
Standardized Features:
```

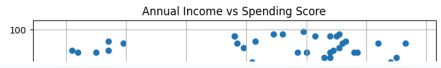
	Annual Income (Standardized)	Spending Score (Standardized)
0	-1.840510	-0.570968
1	-1.781244	1.055427
2	-1.751611	0.640177
3	-1.721978	-1.747509
4	-1.721978	-1.436072

[6]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

plt.figure(figsize=(8, 5))
plt.scatter(X['Annual Income (k$)'],
            X['Spending Score (1-100)'])
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Annual Income vs Spending Score')
plt.grid(True)
plt.show()

sns.pairplot(X)
plt.show()
```



Variables Terminal

Python 3

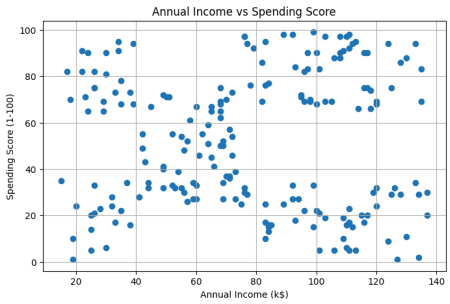
Untitled17.ipynb

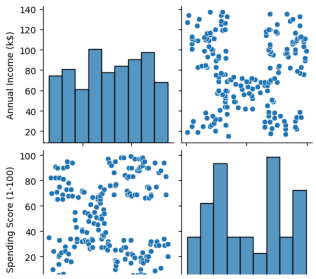
File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk





Variables Terminal Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

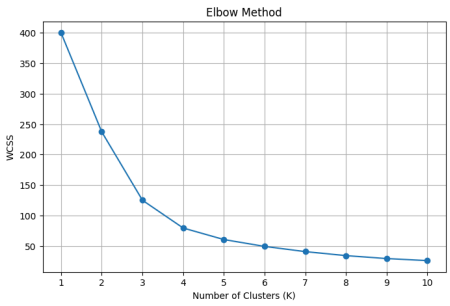
[7]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

wcss = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```



[8]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195
```

Variables Terminal Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[8]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

print("Cluster Labels Added:")
display(df.head())

df.to_csv('Mall_Customers_Clustered.csv', index=False)
files.download('Mall_Customers_Clustered.csv')
```

```
Cluster Labels Added:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Female	28	15	35	4
1	2	Female	62	17	82	2
2	3	Male	60	18	70	2
3	4	Male	70	19	1	4
4	5	Male	52	19	10	4

[9]

```
# Name: Palachuru Nandu Priya
# Roll No: 241801195

centers_scaled = kmeans.cluster_centers_
centers = scaler.inverse_transform(centers_scaled)

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    s=80
)
```

Variables Terminal

Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

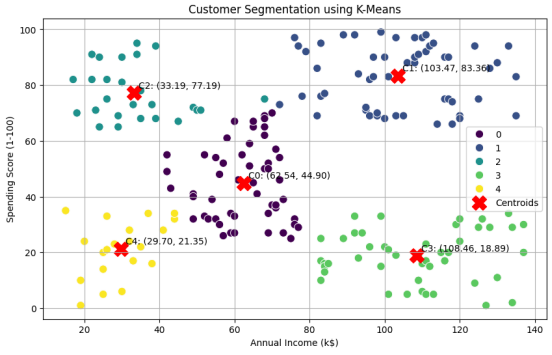
RAM Disk

[9]

```
for i, (income, score) in enumerate(centers):
    plt.annotate(
        f'C{i}: ({income:.2f}, {score:.2f})',
        (income, score),
        xytext=(5, 5),
        textcoords='offset points'
    )

plt.title('Customer Segmentation using K-Means')
plt.legend()
plt.grid(True)
plt.show()

print("Centroid Coordinates:")
for i, (income, score) in enumerate(centers):
    print(f"Cluster {i}: Income = {income:.2f}, "
          f"Spending Score = {score:.2f}")
```




```
Centroid Coordinates:
```

Variables Terminal

Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
[10] # Name: Palachuru Nandu Priya
     # Roll No: 241801195

     print("Number of Customers in Each Cluster:")
     print(df['Cluster'].value_counts().sort_index())

     print("\nAverage Income and Spending Score:")
     cluster_summary = df.groupby('Cluster')[
         ['Annual Income (k$)', 'Spending Score (1-100)']
     ].mean().round(2)

     display(cluster_summary)

     print("\nMarketing Recommendations:")

     for cluster, row in cluster_summary.iterrows():
         income = row['Annual Income (k$)']
         score = row['Spending Score (1-100)']

         if income >= 60 and score >= 60:
             strategy = "Premium offers, loyalty rewards, and exclusive products."
         elif income >= 60 and score < 40:
             strategy = "Personalized promotions to encourage more spending."
         elif income < 40 and score >= 60:
             strategy = "Affordable deals, discounts, and value bundles."
         elif income < 40 and score < 40:
             strategy = "Budget-friendly offers and basic product promotions."
         else:
             strategy = "Targeted discounts and personalized recommendations."

         print(f"Cluster {cluster}: {strategy}")
```

```
Number of Customers in Each Cluster:
Cluster
0    52
1    55
2    27
3    46
4    20
Name: count, dtype: int64

Average Income and Spending Score:
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	62.54	44.90

Variables Terminal Python 3

Untitled17.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
[10]     income = row['Annual Income (k$)']
         score = row['Spending Score (1-100)']

         if income >= 60 and score >= 60:
             strategy = "Premium offers, loyalty rewards, and exclusive products."
         elif income >= 60 and score < 40:
             strategy = "Personalized promotions to encourage more spending."
         elif income < 40 and score >= 60:
             strategy = "Affordable deals, discounts, and value bundles."
         elif income < 40 and score < 40:
             strategy = "Budget-friendly offers and basic product promotions."
         else:
             strategy = "Targeted discounts and personalized recommendations."

         print(f"Cluster {cluster}: {strategy}")
```

```
Number of Customers in Each Cluster:
Cluster
0    52
1    55
2    27
3    46
4    20
Name: count, dtype: int64

Average Income and Spending Score:
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	62.54	44.90
1	103.47	83.36
2	33.19	77.19
3	108.46	18.89
4	29.70	21.35

```
Marketing Recommendations:
Cluster 0: Targeted discounts and personalized recommendations.
Cluster 1: Premium offers, loyalty rewards, and exclusive products.
Cluster 2: Affordable deals, discounts, and value bundles.
Cluster 3: Personalized promotions to encourage more spending.
Cluster 4: Budget-friendly offers and basic product promotions.
```

Variables Terminal Python 3